Node Depths

AlgoExpert · Easy · Binary Trees

The problem

- Sum of the depths (distance from root) of every node.

Intuition

- Each node contributes its depth, so pass the depth down while traversing and add it up.

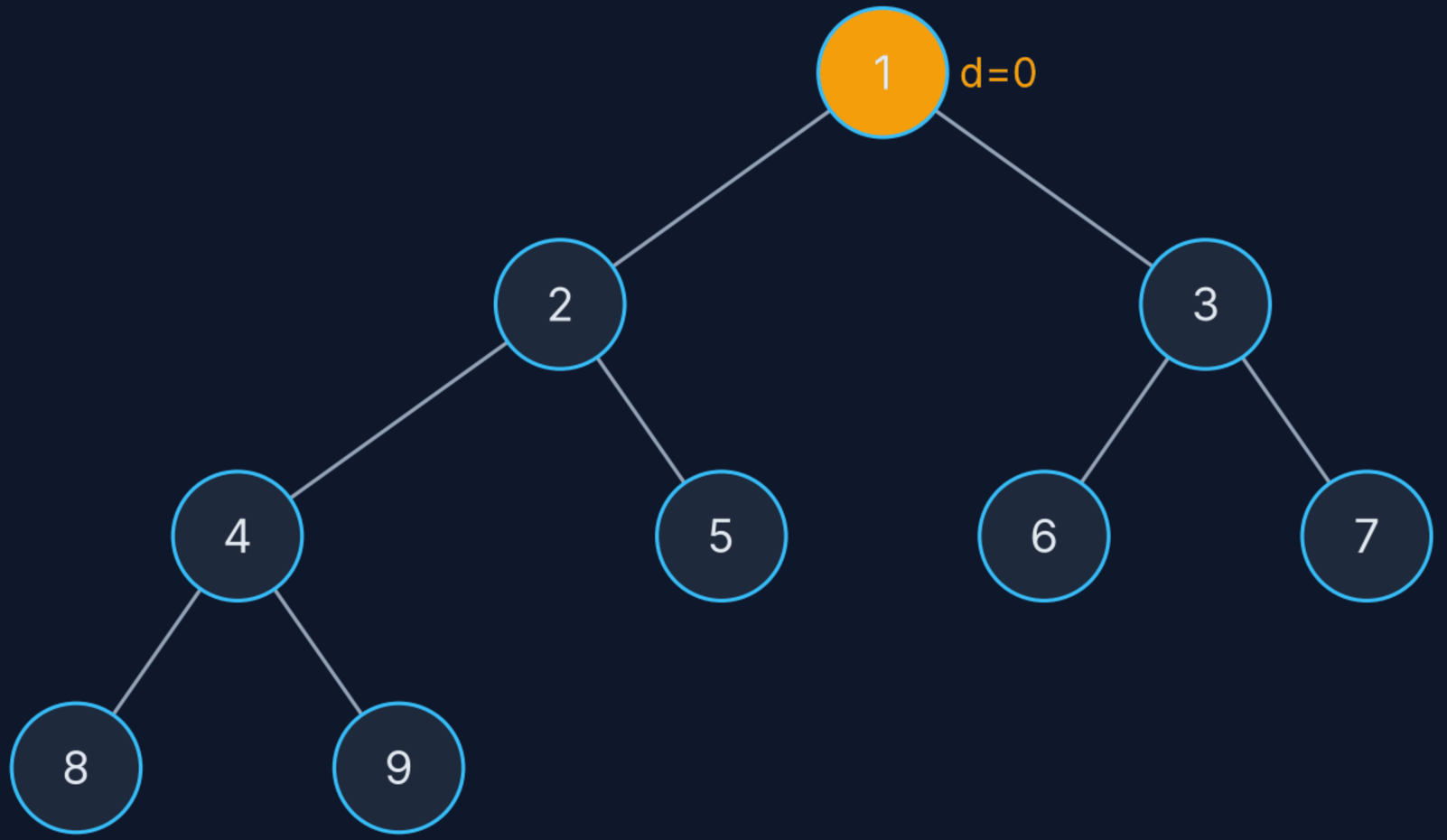
Brute force vs optimal

- Naive: compute each node's depth by walking up from it → O(n·h)
- Optimal: pass depth down in one traversal → O(n)
- BFS by levels also works: depth × level size

Approach

- 1. Traverse from the root at depth zero
- 2. Add the current depth to the total
- 3. Visit children with depth plus one
- 4. Return the sum

Worked example



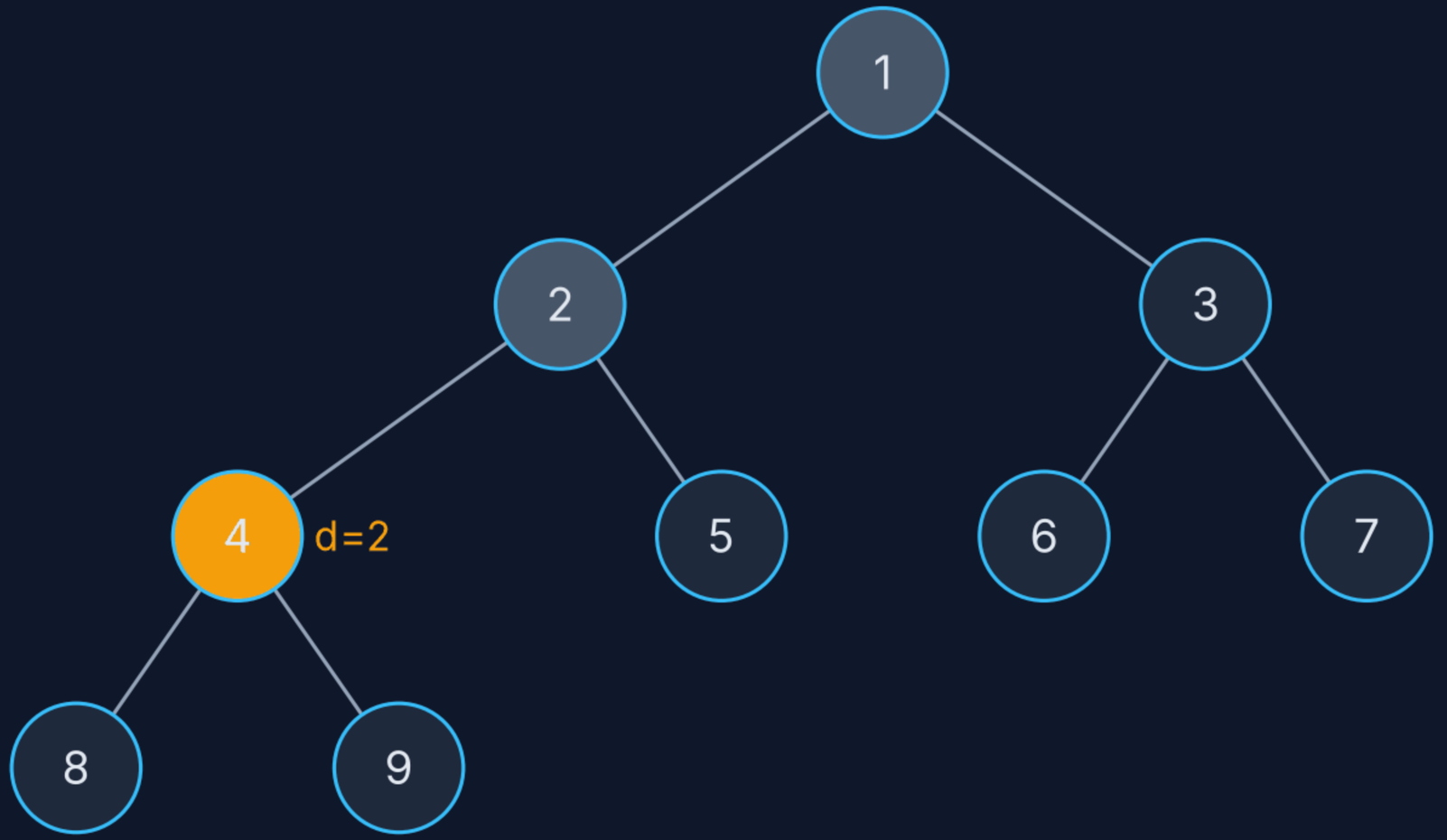

```
node 1

depth 0

total = 0
```

Node 1 at depth 0 → total 0

Worked example



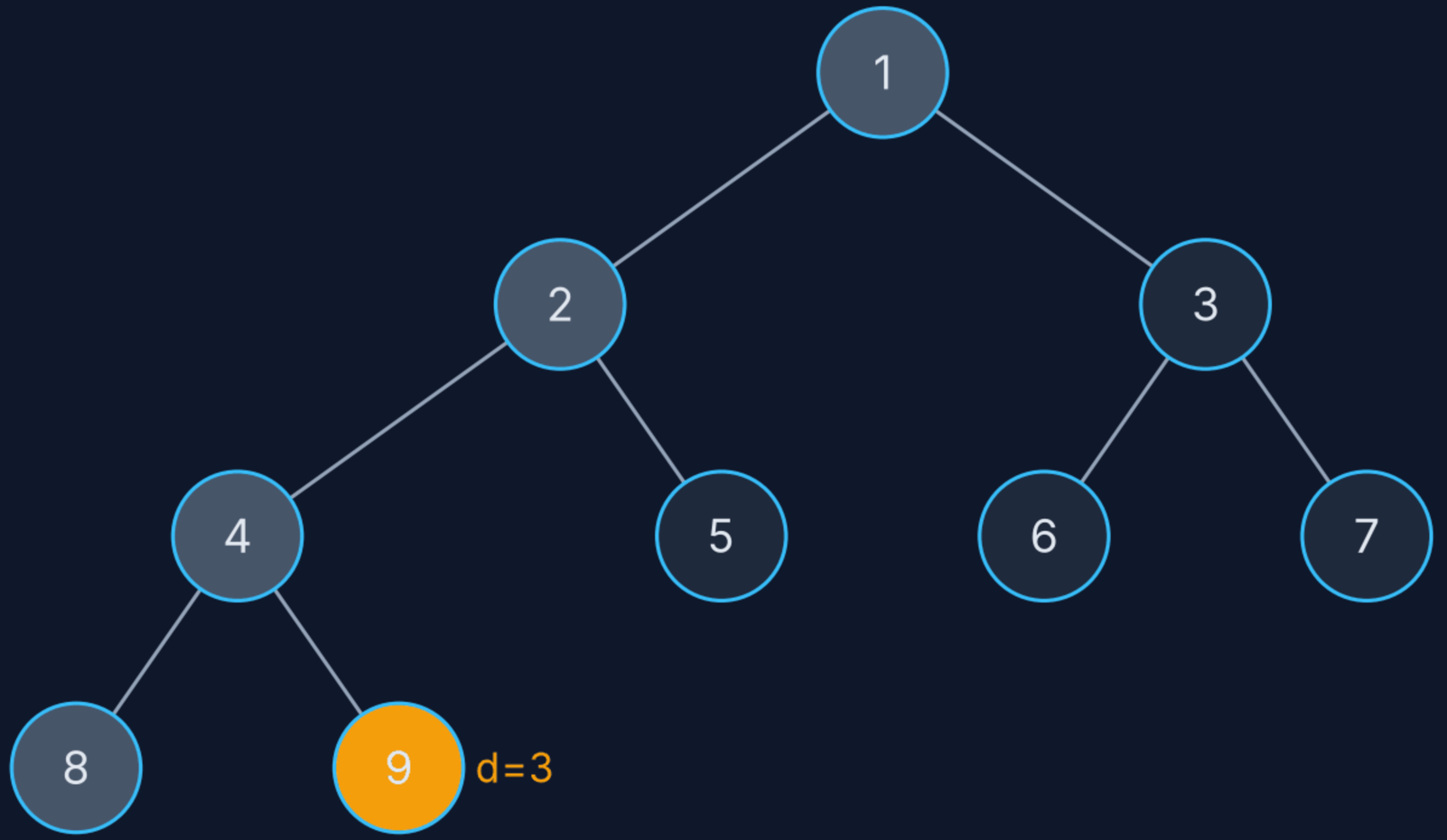

```
node 4

depth 2

total = 3
```

Node 4 at depth 2 → total 3

Worked example



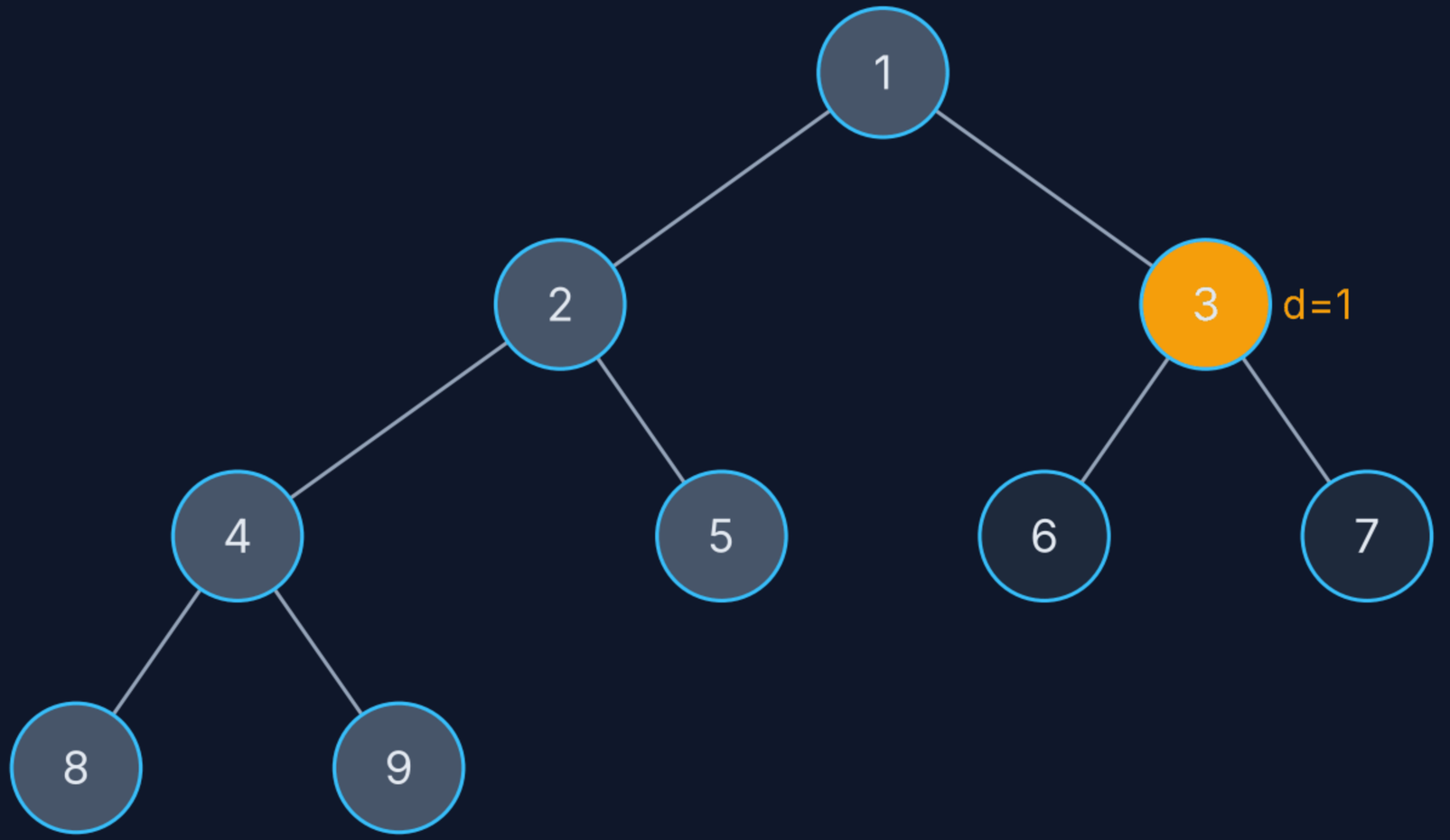

```
node 9

depth 3

total = 9
```

Node 9 at depth 3 → total 9

Worked example



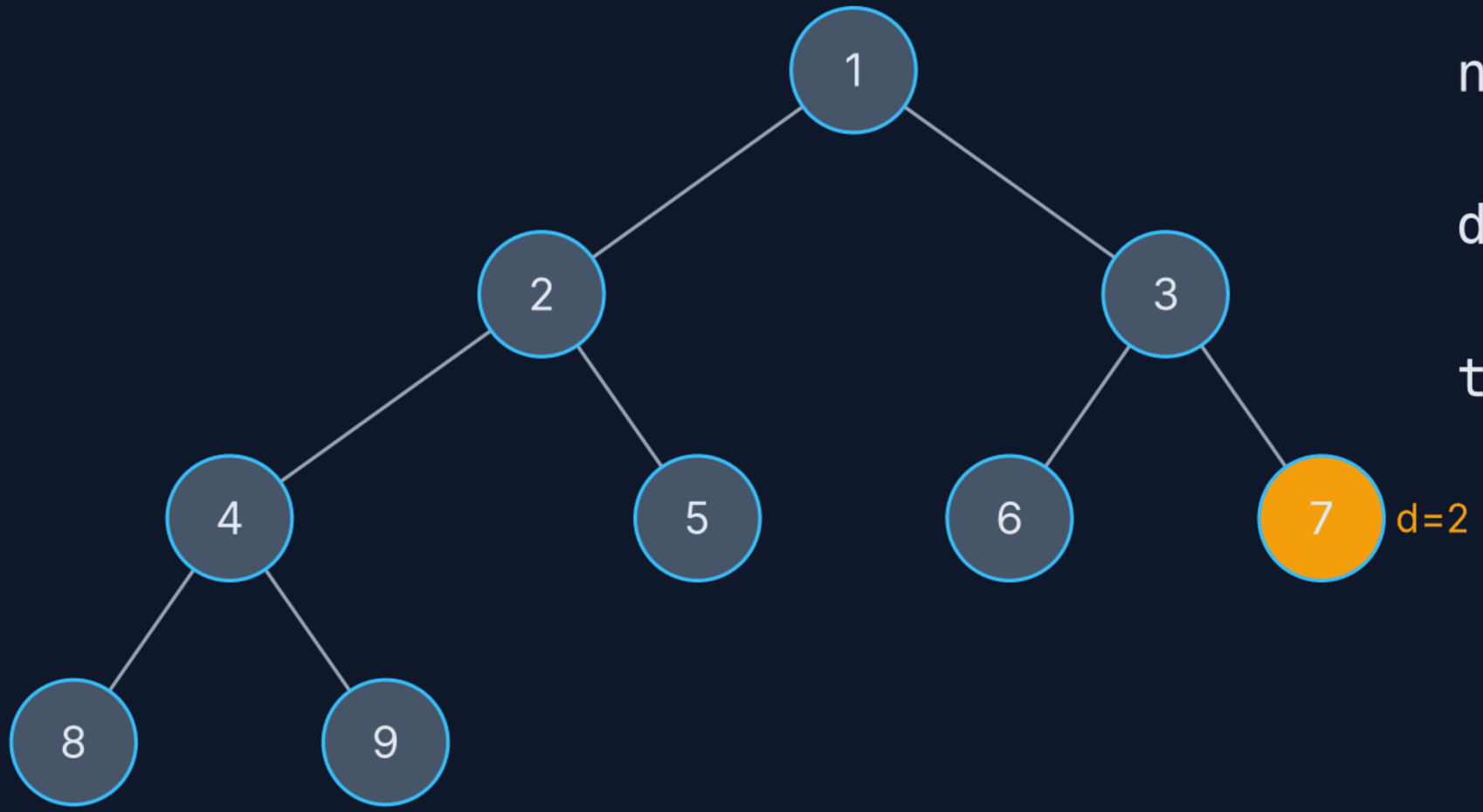

```
node 3

depth 1

total = 12
```

Node 3 at depth 1 → total 12

Worked example




```
node 7

depth 2

total = 16
```

Node 7 at depth 2 → total 16

Java solution

```java
public class NodeDepths {
    public static int nodeDepths(BinaryTree root) { return sum(root, 0); }

    private static int sum(BinaryTree n, int depth) {
        if (n == null) return 0;
        return depth + sum(n.left, depth + 1) + sum(n.right, depth + 1);
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Edge cases & pitfalls

- Root has depth 0
- Empty tree → 0
- Deep trees: recursion depth = height

Interview tips & follow-ups

- Mention iterative stack/queue version
- Follow-up: sum of depths for every subtree (All Kinds Of Node Depths)

Recap

- Each node contributes its depth, so pass the depth down while traversing and add it up.
- Complexity: O(n)